Informe técnico: comparación entre clasificadores clásicos y modelos QML para galaxias XMP externas

Proyecto Quantum

1 de julio de 2026

Resumen

Este trabajo comparó algoritmos clásicos de aprendizaje automático y modelos de aprendizaje automático cuántico con el objetivo específico de evaluar si estos últimos pueden alcanzar una ventaja frente a los primeros en la identificación supervisada de galaxias extremadamente pobres en metales (XMP) recuperadas desde catálogos externos y asociadas a datos espectroscópicos de SDSS. El resultado principal es negativo en el espacio completo de variables: usando las diez features físicas disponibles, los modelos clásicos, especialmente Random Forest e HistGradientBoosting, superan a los quantum kernels probados. Sin embargo, cuando el problema se restringe a subconjuntos físicos compactos, en particular `core_4` y `metallicity_5`, los Quantum Kernel SVC muestran un desempeño comparable y una mejora pequeña pero frecuente en PR-AUC. Esta mejora no alcanza el criterio estricto para declarar una ventaja QML robusta. La conclusión adecuada es más limitada: QML no domina globalmente, pero puede ser competitivo en espacios de baja dimensión construidos con variables físicamente informativas.

Índice

1. Pregunta operacional **3**

2. Construcción de la muestra externa **3**
2.1. Descubrimiento de catálogos externos . . . 3
2.2. Normalización de catálogos . . . 4

3. Recuperación en SDSS y construcción del dataset supervisado **5**
3.1. Crossmatch inicial contra la muestra SDSS previa . . . 5
3.2. Consulta directa a SDSS . . . 5
3.3. Dataset supervisado con controles pareados . . . 5

4. Variables físicas usadas por los clasificadores **6**

5. Modelos comparados **7**
5.1. Modelos clásicos . . . 7
5.2. Modelos QML . . . 7

6. Métricas de evaluación **8**

7. Benchmark principal: espacio completo de features **8**

8. Control metodológico: efecto de usar controles aleatorios **9**

9. Hipótesis de espacios físicos compactos **10**

10. Validación repetida de la señal compacta **11**

11.Métricas top-k y relevancia para seguimiento astronómico **12**

12.Prueba VQC y decisión de no conservarla como línea principal **13**

13.Interpretación final **13**

A. Resumen de comandos usados **14**

1. Pregunta operacional

La pregunta de trabajo fue directa: construir un benchmark supervisado con positivos XMP externos recuperados en SDSS y controles SDSS, y usarlo para comparar clasificadores clásicos contra modelos QML. La comparación no se diseñó para maximizar una métrica artificial, sino para verificar si los modelos QML podían mejorar o al menos igualar a los modelos clásicos en una tarea astronómica concreta: separar objetos externos tipo XMP/metal-poor de controles SDSS no etiquetados como XMP externos.

La secuencia completa fue:

`catálogos externos` → `normalización` → `recuperación en SDSS` → `dataset supervisado` → `benchmark ML/QML` → `análisis por subconjuntos físicos` → `prueba confirmatoria`

La Figura 1 resume esta secuencia y el embudo de objetos que sobreviven hasta el dataset supervisado final. El punto crítico es que no se entrenó directamente sobre todos los objetos externos encontrados, sino sólo sobre aquellos que pudieron conectarse con información espectroscópica y features físicas válidas en SDSS.

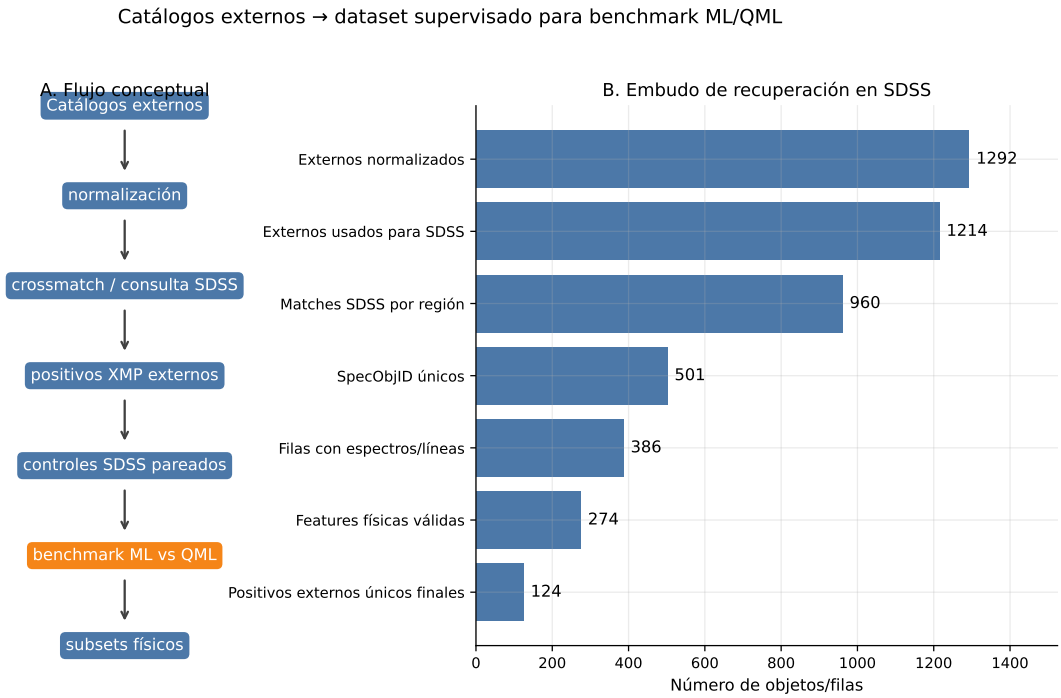


Figura 1: Flujo conceptual y embudo de recuperación de objetos externos en SDSS. Los conteos muestran la reducción desde catálogos externos normalizados hasta positivos XMP externos únicos con features físicas válidas.

2. Construcción de la muestra externa

2.1. Descubrimiento de catálogos externos

El primer paso fue buscar catálogos externos de galaxias raras o extremas. Se usaron búsquedas en VizieR mediante keywords científicas asociadas a objetos Green Pea, galaxias extremadamente pobres en metales, objetos de alta excitación o candidatos LyC/LAE. El objetivo no era todavía entrenar modelos, sino construir una base auditable de objetos externos.

El script usado fue:

```
python3 01_discover_external_rare_galaxy_catalogs_v1.py --max-catalogs-per-keyword 5
    --timeout 60
```

Este script genera un inventario de catálogos, descarga tablas crudas cuando es posible e identifica columnas potencialmente relevantes: coordenadas, redshift, metalicidad, líneas nebulares, índices como O32/R23, equivalent widths e identificadores. La corrida actual produjo 18 catálogos inventariados y 340 registros de columnas. La salida quedó en `results_external_catalog_discovery_v1/`.

2.2. Normalización de catálogos

Los catálogos externos no comparten exactamente el mismo formato. Por tanto, el segundo paso fue normalizarlos a una tabla homogénea con coordenadas en grados, clase tentativa, redshift, metalicidad, masa estelar, O32 y EW(Hα) cuando existieran.

El comando usado fue:

```
python3 02_normalize_external_rare_galaxy_catalogs_v1.py
```

La tabla principal generada fue `external_rare_galaxy_catalogs_normalized_v1.csv`. La corrida produjo 1292 objetos normalizados, todos usables para crossmatch por coordenadas. De ellos, 417 tenían redshift, 776 tenían metalicidad y 43 tenían O32. La composición por clase tentativa se muestra en la Figura 2: 58 Green Peas, 1156 XMP o metal-poor, y 78 candidatos LyC/LAE.

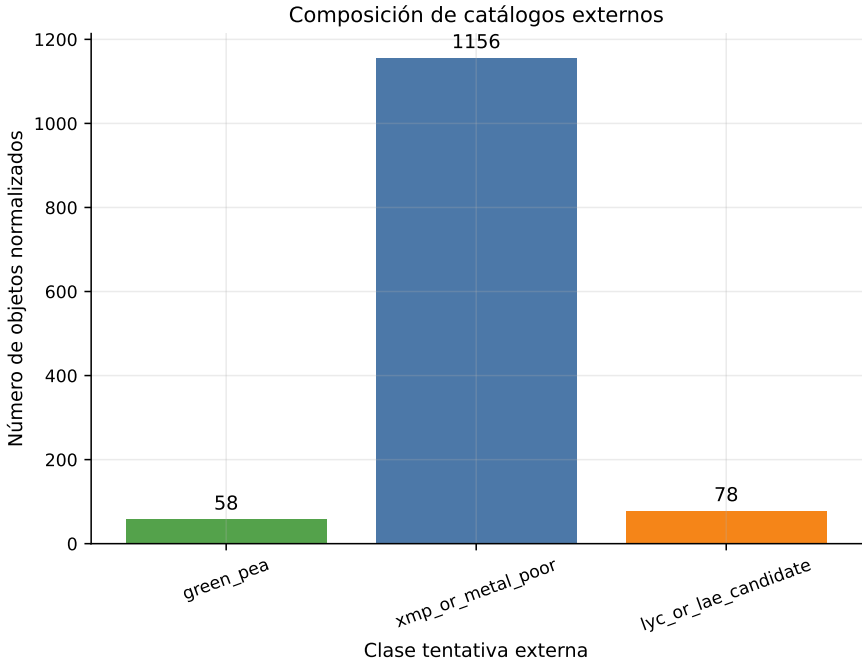


Figura 2: Composición de los catálogos externos normalizados por clase tentativa. La clase dominante es `xmp_or_metal_poor`, que define la población externa principal usada posteriormente.

Una decisión metodológica importante fue conservar inicialmente los candidatos LyC/LAE por trazabilidad, pero excluirlos de la recuperación local en SDSS cuando fuera necesario. Muchos de estos objetos corresponden a alto redshift o sistemas con lente gravitacional y no son comparables directamente con la muestra SDSS local restringida a $0{,}02 < z < 0{,}10$.

3. Recuperación en SDSS y construcción del dataset supervisado

3.1. Crossmatch inicial contra la muestra SDSS previa

El primer intento de asociación con SDSS cruzó la tabla externa contra una muestra SDSS previa de 10.000 galaxias con features físicas. El comando fue:

```
python3 03_crossmatch_external_rare_catalogs_with_sdss_v1.py
```

Este crossmatch usó coordenadas y radio inicial de 2 arcsec. Excluyó por defecto la clase `lyc_or_lae_candidate`. El resultado fue insuficiente para un benchmark supervisado robusto: sólo se obtuvieron 30 matches coordenados y 17 objetos SDSS únicos etiquetados como raros externos. Esto motivó una estrategia más directa: consultar SDSS alrededor de cada objeto externo, no limitarse a la muestra previa de 10.000 galaxias.

3.2. Consulta directa a SDSS

El script de consulta directa recuperó objetos SDSS alrededor de las posiciones externas y luego consultó espectroscopia, líneas nebulares, errores, masa estelar, clase BPT y metalicidad MPA-JHU. El comando completo usado fue:

```
python3 04_query_sdss_for_external_rare_objects_v1.py --timeout 180 --sleep-sec 0.05
    --checkpoint-every 50 --batch-size 5
```

La bitácora también dejó comandos de recuperación operativa, relevantes si una corrida se interrumpe:

```
python3 04_query_sdss_for_external_rare_objects_v1.py --timeout 180 --sleep-sec 0.05
    --checkpoint-every 50 --batch-size 5 --resume

python3 04_query_sdss_for_external_rare_objects_v1.py --reuse-region-matches --batch-
    size 5 --timeout 180
```

La corrida corregida reutilizando matches de región produjo 1214 objetos externos consultados, 960 matches SDSS por región, 501 `SpecObjID` únicos, 386 filas con espectros/líneas y 274 filas con features físicas válidas. El filtrado de valores inválidos, por ejemplo `metallicity = -9999`, fue necesario antes de construir el dataset final.

3.3. Dataset supervisado con controles pareados

El dataset supervisado se construyó con positivos externos XMP recuperados en SDSS y controles SDSS. El comando fue:

```
python3 05_build_external_xmp_supervised_dataset_v1.py
```

Este script produjo dos datasets:

- **Dataset B, pareado**: positivos externos y controles SDSS pareados en redshift z y masa estelar $\log M_\star$. Es el benchmark científicamente relevante.
- **Dataset A, aleatorio**: positivos externos y controles SDSS aleatorios. Se conserva sólo como control metodológico para mostrar qué ocurre cuando no se controla el sesgo de selección.

La corrida final produjo 124 positivos externos únicos y un pool de 9984 controles SDSS. Tanto el dataset pareado como el aleatorio tienen 744 filas: 124 positivos y 620 controles. La Figura 3 muestra por qué el dataset pareado debe ser el benchmark principal. En el dataset pareado, las medianas de z y masa entre positivos y controles son similares. En el dataset aleatorio, los controles tienen redshift y masa desplazados, lo que permite que los modelos aprendan diferencias de selección en vez de diferencias físicas asociadas a la clase XMP.

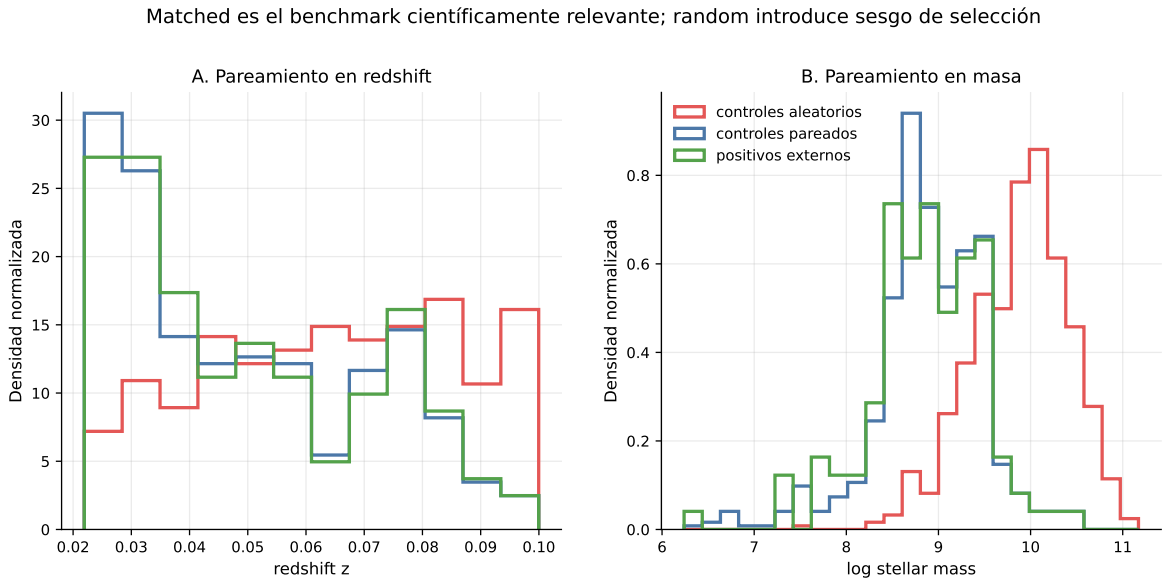


Figura 3: Comparación de positivos externos, controles pareados y controles aleatorios en redshift y masa estelar. El pareamiento evita que el clasificador use redshift o masa como atajos espurios.

Los valores resumidos fueron:

Muestra	Mediana z	Mediana $\log M_\star$
Positivos externos, dataset pareado	0.0428	8.886
Controles pareados	0.0455	8.895
Controles aleatorios	0.0659	9.939

4. Variables físicas usadas por los clasificadores

Las variables del benchmark son transformaciones de líneas nebulares, masa estelar y redshift. Se usan logaritmos de ratios porque las intensidades de líneas cubren rangos amplios y porque muchos diagnósticos de galaxias star-forming se expresan naturalmente en el plano logarítmico.

Tabla 1: Variables principales usadas en los modelos.

Variable	Significado operativo
`log_nii_ha`	$\log([\mathrm{NII}]/\mathrm{H}\alpha)$. Es sensible a enriquecimiento químico, especialmente nitrógeno, y se usa en diagnósticos de metalicidad como N2. Valores bajos tienden a asociarse con baja metalicidad.
`log_oiii_hb`	$\log([\mathrm{OIII}]/\mathrm{H}\beta)$. Traza excitación e ionización. Galaxias compactas, jóvenes o pobres en metales pueden mostrar valores altos.
`log_oii_hb`	$\log([\mathrm{OII}]/\mathrm{H}\beta)$. Complementa la información de oxígeno ionizado y ayuda a construir R23 y O32.
`log_sii_ha`	$\log([\mathrm{SII}]/\mathrm{H}\alpha)$. Se usa como diagnóstico nebular adicional y ayuda a distinguir condiciones de ionización.
`log_r23`	$\log(([\mathrm{OII}]+[\mathrm{OIII}])/\mathrm{H}\beta)$. R23 es un indicador clásico relacionado con abundancia de oxígeno, aunque puede ser degenerado con metalicidad.
`log_o32`	$\log([\mathrm{OIII}]/[\mathrm{OII}])$. Es sensible al parámetro de ionización y a la dureza del campo ionizante.

Variable	Significado operativo
o3n2	Combinación de [OIII]/Hβ y [NII]/Hα. Resume diferencias entre excitación de oxígeno y enriquecimiento por nitrógeno.
Z_N2_PP04	Estimador de metalicidad derivado del índice N2.
Z_O3N2_PP04	Estimador de metalicidad derivado del índice O3N2.
delta_N2_O3N2	Diferencia entre los estimadores N2 y O3N2. Es una medida interna de discrepancia diagnóstica; puede capturar objetos con condiciones físicas no estándar.
log_stellar_mass	Logaritmo de la masa estelar. Es una variable global de la galaxia, no sólo del gas ionizado.
z	Redshift espectroscópico. Se usó para pareamiento, pero puede introducir sesgos si se deja que el modelo lo use para separar muestras con distinta selección observacional.

No se pareó en metalicidad ni en ratios nebulares. Esa decisión es deliberada: esas variables contienen precisamente la señal física que el clasificador debe poder explotar. En cambio, sí se pareó en redshift y masa porque esas dos variables pueden funcionar como atajos de selección si los controles no se construyen cuidadosamente.

5. Modelos comparados

5.1. Modelos clásicos

Los modelos clásicos usados representan familias diferentes de clasificadores. Esto es importante porque no basta con comparar QML contra un único baseline débil.

- **Logistic Regression**. Es un clasificador lineal. Aprende una frontera de decisión lineal en el espacio de features transformadas. Es útil como baseline simple: si funciona bien, la separación entre clases es aproximadamente lineal o está fuertemente dominada por pocas variables.
- **Random Forest**. Es un ensamble de árboles de decisión entrenados sobre subconjuntos de datos y variables. Captura relaciones no lineales e interacciones entre features sin requerir escalado complejo. Suele ser fuerte en tablas con variables heterogéneas.
- **HistGradientBoosting**. Es un modelo de boosting basado en árboles. Construye árboles secuencialmente, donde cada árbol intenta corregir errores del conjunto anterior. Suele capturar estructura no lineal de forma eficiente.
- **SVC con kernel RBF**. Es una máquina de soporte vectorial con un kernel radial. Proyecta implícitamente los datos a un espacio de alta dimensión donde fronteras no lineales pueden volverse separables.
- **SVC con kernel polinomial**. Es una SVC que permite interacciones polinomiales entre variables. Es útil cuando la frontera de decisión puede aproximarse mediante combinaciones polinomiales de las features.

5.2. Modelos QML

Los modelos QML evaluados se basan principalmente en kernels cuánticos.

- **Quantum Kernel SVC reuploading**. Primero codifica las features clásicas en un circuito cuántico parametrizado. El circuito define un mapa de features cuántico. Luego se calcula una matriz de similitud, o kernel, entre pares de galaxias. Esa matriz se usa dentro de una SVC

clásica. La parte cuántica no decide directamente la clase; construye una noción de similitud potencialmente distinta de la que producen kernels clásicos como RBF o polinomial.

- **Quantum Kernel SVC ZZ-like**. Variante de mapa de features cuántico con estructura tipo ZZ. En estas corridas resultó claramente peor que el reuploading y, además, la implementación usada fue determinística respecto a la semilla, por lo que las semillas repetidas no representan realizaciones independientes.
- **Variational Quantum Classifier**. Se implementó una prueba compacta posterior sobre `core_4` y `metallicity_5`. A diferencia del kernel cuántico, el VQC entrena parámetros de un circuito variacional para clasificar directamente. La prueba corta no mejoró los resultados: quedó por debajo del mejor clásico en PR-AUC. Por eso no se conserva como línea prometedora ni se generó figura final para esa prueba.

En términos prácticos, el resultado QML relevante de este análisis no proviene de VQC, sino de Quantum Kernel SVC con feature map reuploading.

6. Métricas de evaluación

La métrica principal fue PR-AUC, no accuracy. Esto es importante porque el problema es de clase rara: los positivos XMP externos son minoría frente a los controles. En problemas desbalanceados, accuracy puede ser engañosa porque un modelo puede acertar muchos negativos y aun así ser pobre para recuperar positivos.

- **PR-AUC**. Área bajo la curva precision–recall. Mide cuán bien el modelo recupera positivos manteniendo precisión alta. Es la métrica primaria.
- **ROC-AUC**. Área bajo la curva ROC. Mide separación global entre positivos y negativos, pero puede verse menos afectada por desbalance que PR-AUC. Se usa como métrica secundaria.
- **precision@20 y precision@50**. Fracción de verdaderos positivos entre los 20 o 50 objetos mejor rankeados por el modelo. Es relevante para astronomía porque el seguimiento observacional suele priorizar listas cortas de candidatos.
- **recall_at_top5percent**. Fracción de positivos recuperada dentro del 5 % superior del ranking.
- **F1 y balanced accuracy con umbral oracle**. Se calcularon como diagnóstico, pero no son métricas principales porque el umbral fue elegido usando el fold de test. Eso las convierte en medidas post-hoc, no en estimadores limpios de rendimiento operacional.

7. Benchmark principal: espacio completo de features

El benchmark ML/QML inicial se corrió con el dataset pareado y todas las features físicas disponibles. Primero se usó una prueba rápida clásica:

```
python3 06_compare_external_xmp_qml_classical_v1.py --dataset matched --n-folds 3 --
    skip-quantum
```

La corrida completa recomendada con QML fue:

```
python3 06_compare_external_xmp_qml_classical_v1.py --dataset matched --n-folds 5 --
    quantum-feature-maps reuploading,zz --quantum-layers 2 --quantum-seeds 101,202,303
```

También se corrió sobre controles aleatorios como control metodológico:

```
python3 06_compare_external_xmp_qml_classical_v1.py --dataset random --n-folds 5 --
    quantum-feature-maps reuploading,zz --quantum-layers 2 --quantum-seeds 101,202,303
```

La Figura 4 muestra el resultado principal para el dataset pareado. En el espacio completo, el mejor modelo fue Random Forest, con PR-AUC $\simeq$ 0,544 y ROC-AUC $\simeq$ 0,831. HistGradient-Boosting y SVC polinomial quedaron cerca. El mejor QK-SVC reuploading quedó alrededor de PR-AUC $\simeq$ 0,48. Por tanto, no hay ventaja QML en el espacio completo de diez features.

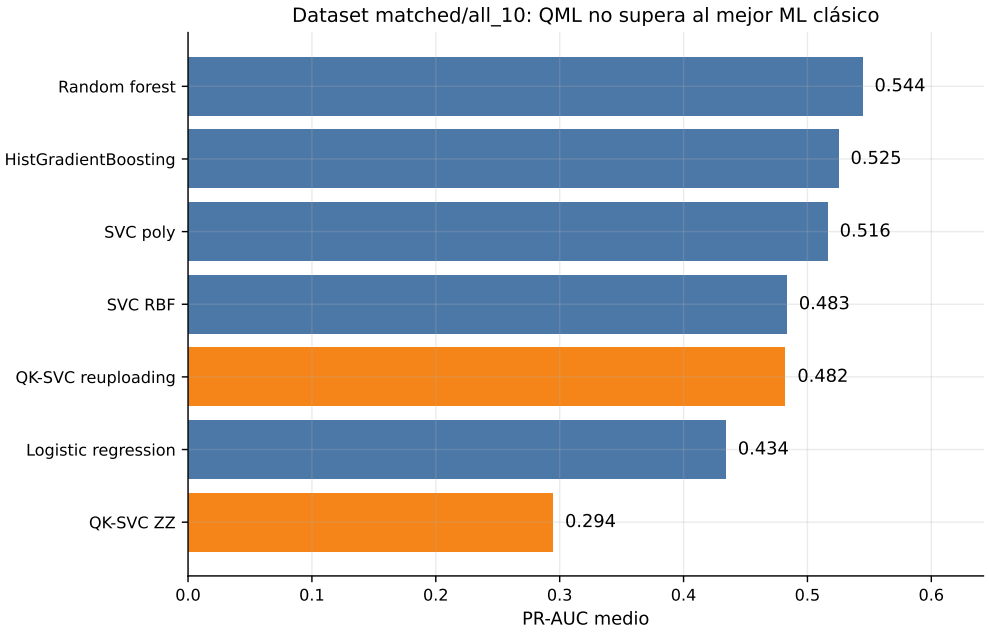


Figura 4: Benchmark principal en el dataset pareado usando el espacio completo de features. La métrica primaria es PR-AUC. Los modelos clásicos superan a los quantum kernels probados.

Modelo	PR-AUC	ROC-AUC
Random Forest	0.544	0.831
HistGradientBoosting	0.525	0.828
SVC polinomial	0.516	0.820
SVC RBF	0.483	0.825
QK-SVC reuploading	0.482	0.825
Logistic Regression	0.434	0.800
QK-SVC ZZ-like	0.294	0.687

8. Control metodológico: efecto de usar controles aleatorios

El dataset con controles aleatorios produjo métricas mucho más altas. Por ejemplo, QK-SVC reuploading alcanzó PR-AUC $\simeq$ 0,84, SVC RBF alcanzó PR-AUC $\simeq$ 0,82 y Logistic Regression alcanzó PR-AUC $\simeq$ 0,81. Esto no debe interpretarse como que el problema sea fácil de forma física. Más bien indica que los controles aleatorios introducen un sesgo fuerte: los modelos pueden separar positivos y negativos usando diferencias de redshift, masa o selección observacional.

La Figura 5 resume esta comparación. La conclusión es metodológica: el benchmark `random` infla el rendimiento y no debe usarse para afirmar desempeño científico. El benchmark `matched` es el relevante.

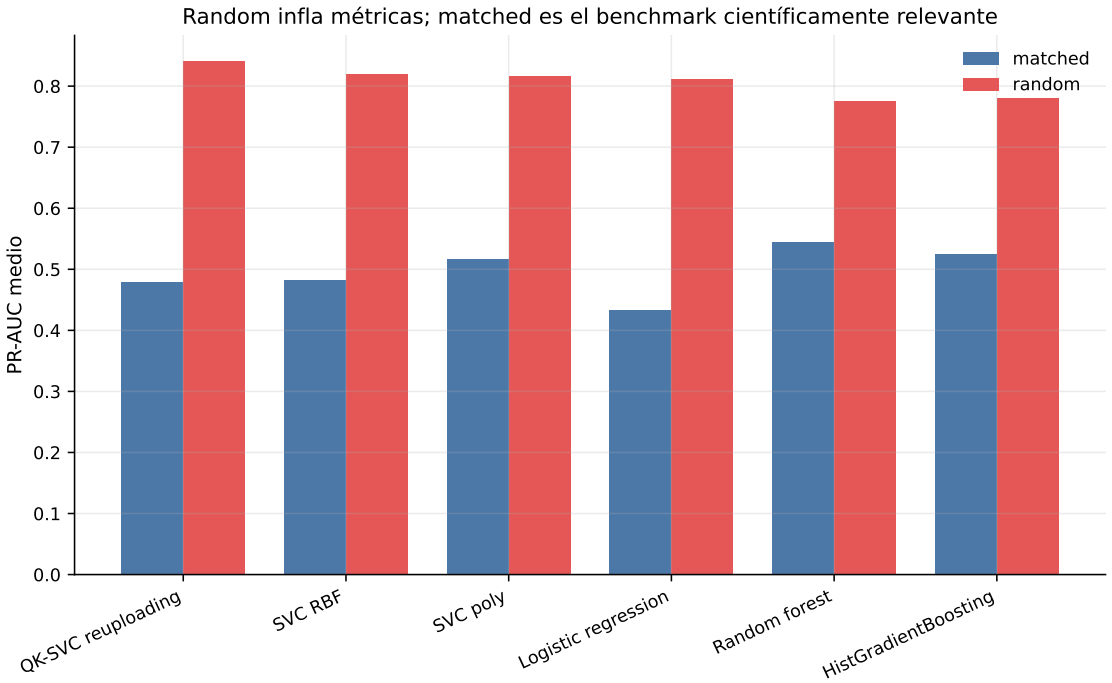


Figura 5: Comparación de PR-AUC entre datasets pareado y aleatorio. El dataset aleatorio infla las métricas por sesgo de selección; el dataset pareado es el benchmark científicamente interpretable.

9. Hipótesis de espacios físicos compactos

Después del resultado negativo en `all_10`, se probó una hipótesis más específica: quizá los kernels cuánticos no son competitivos cuando se les entrega un espacio tabular completo con variables globales, pero sí pueden serlo en espacios compactos, de baja dimensión, construidos con diagnósticos nebulares físicamente motivados.

El barrido se ejecutó con:

```
python3 07_qml_feature_subset_and_hyperparam_scan_v1.py \
  --n-folds 5 \
  --feature-subsets core_4,metallicity_5,oxygen_6,nebular_no_mass_z_8,all_10 \
  --quantum-layers 1,2,3,4,6 \
  --quantum-c-values 0.3,1.0,3.0,10.0,30.0 \
  --quantum-seeds 101,202,303 \
  --top-k 20,50 \
  --output-dir results_external_xmp_qml_feature_scan_v1
```

También se dejó una prueba clásica rápida:

```
python3 07_qml_feature_subset_and_hyperparam_scan_v1.py \
  --skip-quantum \
  --n-folds 3 \
  --feature-subsets core_4 \
  --top-k 20 \
  --output-dir results_external_xmp_qml_feature_scan_check_v1
```

Los subconjuntos fueron:

- `core_4`: `log_nii_ha`, `log_oiii_hb`, `log_o32`, `delta_N2_O3N2`. Es el conjunto mínimo de diagnóstico nebular y discrepancia de metalicidad.
- `metallicity_5`: agrega `log_r23` a `core_4`. Está centrado explícitamente en metalicidad y oxígeno.

- `oxygen_6`: prioriza variables de oxígeno y diagnósticos derivados.
- `nebular_no_mass_z_8`: usa ratios nebulares y diagnósticos derivados, excluyendo masa y redshift.
- `all_10`: espacio completo, incluyendo masa y redshift.

El resultado se resume en la Figura 6. QML mejoró al mejor clásico en `core_4` por +0,0396 en PR-AUC y en `metallicity_5` por +0,0230. En `oxygen_6` el resultado fue prácticamente empate. En `all_10` y `nebular_no_mass_z_8`, QML quedó por debajo del mejor clásico.

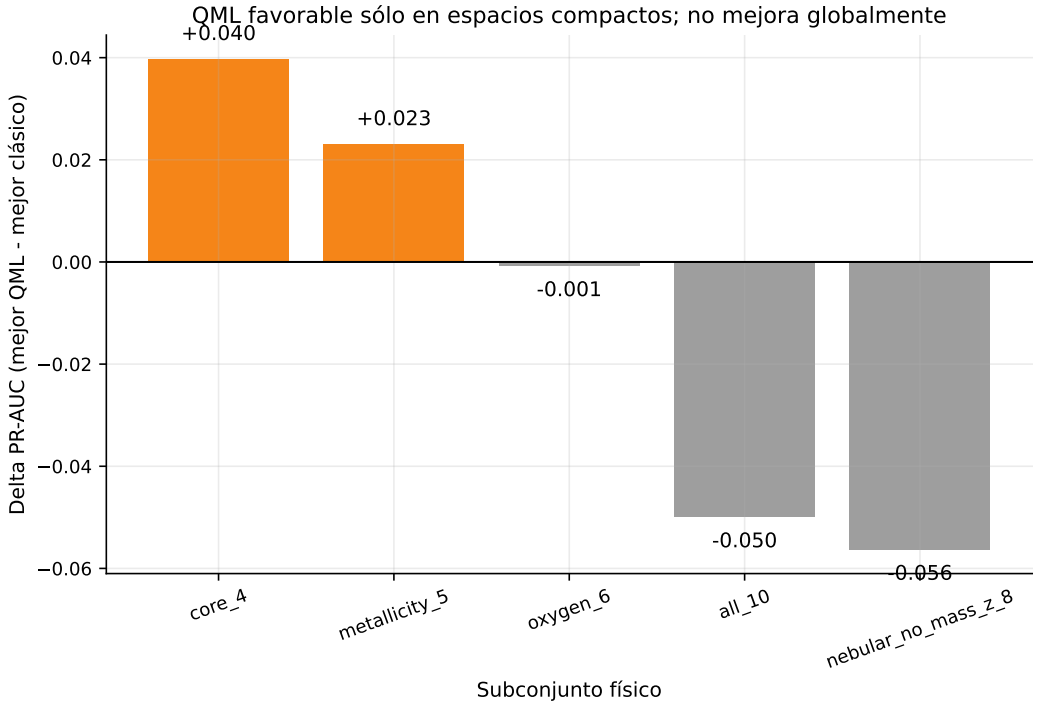


Figura 6: Diferencia de PR-AUC entre el mejor QML y el mejor clásico por subconjunto físico. Los deltas positivos aparecen sólo en espacios compactos, especialmente `core_4` y `metallicity_5`.

El patrón técnico fue que las mejores configuraciones QML usaron pocas capas, típicamente 1–2, y valores altos de C, típicamente 10 o 30. Capas más profundas no mejoraron claramente y probablemente añadieron complejidad sin beneficio.

10. Validación repetida de la señal compacta

El resultado del Script 07 era exploratorio porque surgía de un barrido amplio. Para evitar sobreinterpretarlo, el Script 08 restringió el análisis a `core_4` y `metallicity_5`, acotó el espacio QML y repitió validación cruzada estratificada con múltiples semillas.

El comando completo fue:

```
python3 08_confirm_qml_core_feature_advantage_v1.py --input
    results_external_xmp_supervised_dataset_v1/
    dataset_external_xmp_matched_controls_v1.csv --output-dir
    results_external_xmp_qml_confirm_core_features_v1 --n-folds 5 --cv-seeds
    11,22,33,44,55,66,77,88,99,111 --feature-subsets core_4,metallicity_5 --quantum-
    layers 1,2 --quantum-c-values 10,30 --quantum-seeds
    101,202,303,404,505,606,707,808,909,1001 --top-k 20,50
```

En cada semilla de validación cruzada, el script seleccionó el mejor clásico y el mejor QML, y calculó:

$$\Delta_{\text{QML}-\text{classical}} = \text{PR-AUC}_{\text{mejor QML}} - \text{PR-AUC}_{\text{mejor clásico}}. \tag{1}$$

La Figura 7 muestra la distribución de estos deltas. Para `core_4`, el delta medio fue +0,0187, la mediana fue +0,0181 y la fracción de particiones con delta positivo fue 0.90. Para `metallicity_5`, el delta medio fue +0,0136, la mediana fue +0,0156 y la fracción positiva fue 0.70.

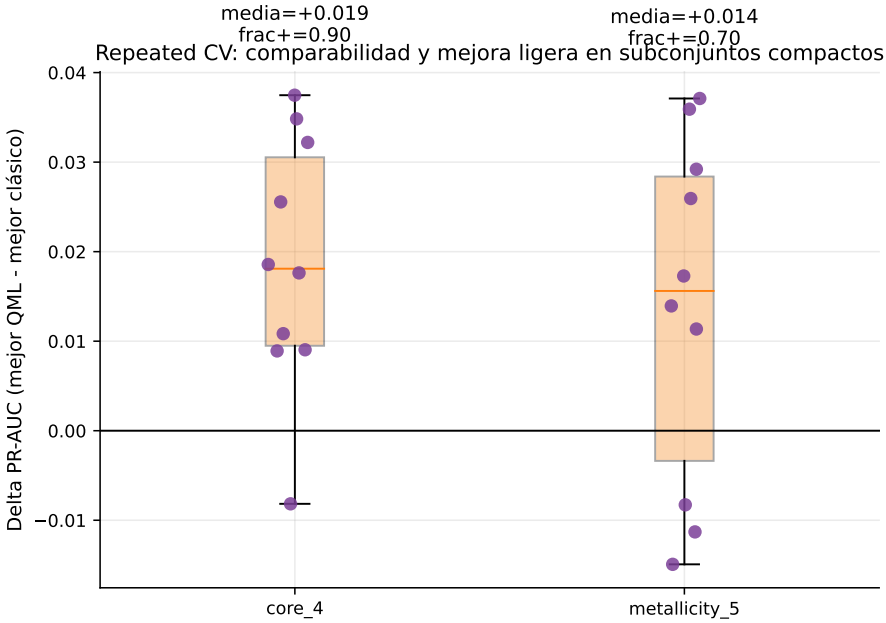


Figura 7: Repeated CV para la señal compacta. Cada punto representa una semilla de partición CV. La señal QML es pequeña pero frecuentemente positiva, especialmente en `core_4`.

El criterio confirmatorio estricto definido en el script requería mediana positiva, media mayor o igual a 0.02, fracción positiva de al menos 0.70 y ausencia de pérdida clara en métricas top-k. Bajo ese criterio, ni `core_4` ni `metallicity_5` alcanzan una ventaja QML robustamente confirmada. La razón principal es que la media de PR-AUC queda por debajo del umbral de 0.02, aunque `core_4` queda muy cerca.

11. Métricas top-k y relevancia para seguimiento astronómico

Además de PR-AUC, se evaluaron métricas top-k porque en astronomía muchas aplicaciones no requieren clasificar todo el catálogo, sino producir una lista corta de candidatos para inspección o seguimiento. La Figura 8 muestra que QML no pierde de forma clara en ranking top-k y puede mejorar ligeramente `precision@20`.

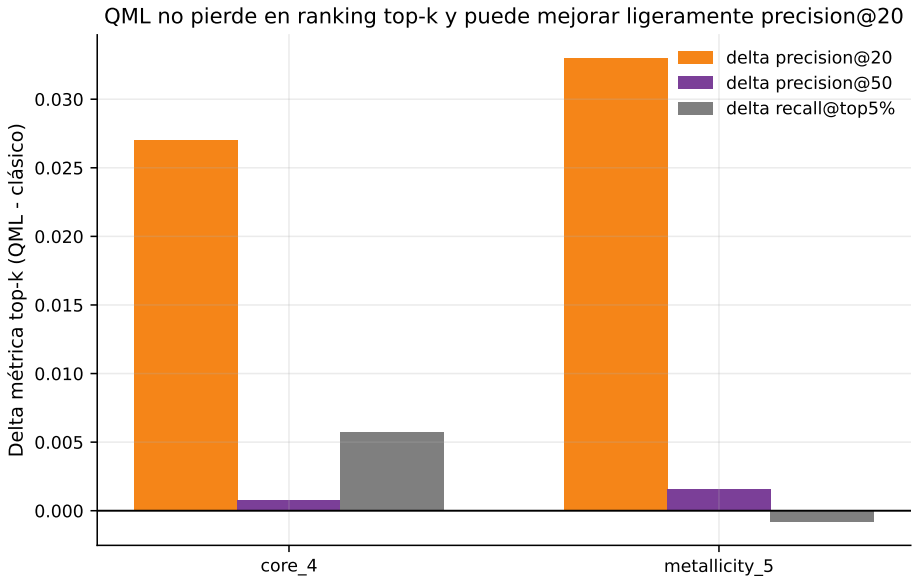


Figura 8: Deltas QML–clásico en métricas top-k para los subconjuntos compactos. La mejora más visible aparece en `precision@20`; las diferencias en `precision@50` y `recall_at_top5percent` son muy pequeñas.

Los valores medios fueron:

Subset	Δ precision@20	Δ precision@50	Δ recall@top5 %
`core_4`	$+0.0270$	$+0.0008$	$+0.0057$
`metallicity_5`	$+0.0330$	$+0.0016$	$-0{,}0008$

La lectura debe ser prudente: las mejoras top-k son pequeñas. No justifican afirmar una ventaja operacional fuerte, pero sí son consistentes con la idea de que QK-SVC reuploading no queda rezagado en los subconjuntos compactos y puede ordenar ligeramente mejor los primeros candidatos.

12. Prueba VQC y decisión de no conservarla como línea principal

Se implementó una prueba compacta con un Variational Quantum Classifier sobre `core_4` y `metallicity_5`. El VQC entrena directamente parámetros de un circuito cuántico para producir la clasificación, a diferencia del quantum kernel, que sólo usa el circuito para construir similitudes entre objetos.

La corrida corta no mejoró los resultados: el VQC quedó claramente por debajo del mejor clásico en PR-AUC y también perdió en métricas top-k. Los deltas aproximados fueron de $\sim -0{,}10$ en PR-AUC para `core_4` y $\sim -0{,}11$ para `metallicity_5`. Por ello, la carpeta de resultados VQC fue eliminada y no se generó figura final asociada. Esta decisión evita sobrecargar el informe con una rama que no aporta a la conclusión principal. La variante QML prometedora dentro de este análisis es QK-SVC reuploading.

13. Interpretación final

La comparación completa lleva a cuatro conclusiones operativas.

1. **El dataset pareado es indispensable.** El control aleatorio infla las métricas porque introduce diferencias de redshift y masa. Las conclusiones científicas deben basarse en el dataset `matched`.

2. **No hay ventaja QML global.** En el espacio completo `all_10`, los modelos clásicos superan a los quantum kernels. El mejor clásico fue Random Forest, con PR-AUC $\simeq 0{,}544$.

3. **Sí aparece una señal QML compacta.** En `core_4` y `metallicity_5`, QK-SVC reuploading supera levemente al mejor clásico en el barrido exploratorio y mantiene deltas frecuentemente positivos en repeated CV.

4. **La señal no es una ventaja robusta.** El tamaño del efecto es pequeño y no supera el criterio confirmatorio estricto. La formulación correcta es "QML competitivo y ligeramente favorable en espacios físicos compactos", no "ventaja cuántica demostrada".

En síntesis, el análisis construyó un benchmark externo auditable de galaxias XMP recuperadas en SDSS, mostró por qué el pareamiento en redshift y masa es necesario, comparó modelos clásicos y QML bajo métricas adecuadas para clase rara, y encontró que los quantum kernels no superan a ML clásico en el espacio completo, pero pueden ser competitivos en subconjuntos físicos compactos. Este resultado es científicamente útil precisamente porque es restrictivo: delimita dónde QML no aporta y dónde podría valer la pena seguir explorando.

A. Resumen de comandos usados

Se listan los comandos relevantes del pipeline. No se incluyen comandos de generación de figuras.

```
python3 01_discover_external_rare_galaxy_catalogs_v1.py --max-catalogs-per-keyword 5
    --timeout 60

python3 02_normalize_external_rare_galaxy_catalogs_v1.py

python3 03_crossmatch_external_rare_catalogs_with_sdss_v1.py

python3 04_query_sdss_for_external_rare_objects_v1.py --timeout 180 --sleep-sec 0.05
    --checkpoint-every 50 --batch-size 5

python3 04_query_sdss_for_external_rare_objects_v1.py --timeout 180 --sleep-sec 0.05
    --checkpoint-every 50 --batch-size 5 --resume

python3 04_query_sdss_for_external_rare_objects_v1.py --reuse-region-matches --batch-
    size 5 --timeout 180

python3 05_build_external_xmp_supervised_dataset_v1.py

python3 06_compare_external_xmp_qml_classical_v1.py --dataset matched --n-folds 3 --
    skip-quantum

python3 06_compare_external_xmp_qml_classical_v1.py --dataset matched --n-folds 5 --
    quantum-feature-maps reuploading,zz --quantum-layers 2 --quantum-seeds 101,202,303

python3 06_compare_external_xmp_qml_classical_v1.py --dataset random --n-folds 5 --
    quantum-feature-maps reuploading,zz --quantum-layers 2 --quantum-seeds 101,202,303

python3 07_qml_feature_subset_and_hyperparam_scan_v1.py \
  --n-folds 5 \
  --feature-subsets core_4,metallicity_5,oxygen_6,nebular_no_mass_z_8,all_10 \
  --quantum-layers 1,2,3,4,6 \
  --quantum-c-values 0.3,1.0,3.0,10.0,30.0 \
  --quantum-seeds 101,202,303 \
  --top-k 20,50 \
```

```
  --output-dir results_external_xmp_qml_feature_scan_v1

python3 07_qml_feature_subset_and_hyperparam_scan_v1.py \
  --skip-quantum \
  --n-folds 3 \
  --feature-subsets core_4 \
  --top-k 20 \
  --output-dir results_external_xmp_qml_feature_scan_check_v1

python3 08_confirm_qml_core_feature_advantage_v1.py --input
    results_external_xmp_supervised_dataset_v1/
    dataset_external_xmp_matched_controls_v1.csv --output-dir
    results_external_xmp_qml_confirm_core_features_v1 --n-folds 5 --cv-seeds
    11,22,33,44,55,66,77,88,99,111 --feature-subsets core_4,metallicity_5 --quantum-
    layers 1,2 --quantum-c-values 10,30 --quantum-seeds
    101,202,303,404,505,606,707,808,909,1001 --top-k 20,50
```